Eduvos Assessment Coversheet

Academic Details

Campus: **Eduvos Online**

Faculty: **Information Technology**

Module Code: **ITBDA4-14**

Lecturer's Name: **Gilbert Lekula**

Group (if applicable) _____

Assessment Details

Assessment Type: **Assignment**

Assignment/Brief Number: _____

Due Date/ Submission Date: **13th November**

Student Details (Group, if applicable)

#	Full Name	Student Number	Contact Number	% Participation	Signature
1					
2					
3					
4					
5					
6					

Student Details (Individual, if applicable)

Student Full Name: **Cristina Abrahamse**

Student Number: **DV.2023.N5K8W1**

Academic Integrity

AI Disclosure Completed [X] **Yes** [ ] **No**

Declaration:
I declare that this assessment is my own original work except for source material explicitly acknowledged. I confirm that this work has not previously been submitted for assessments in any course. I acknowledge the institutional AI policy and confirm that AI tools have only been used as stipulated and documented in the AI Disclosure Appendix.

Student Signature **CEA**

Date **03 Nov 2026**

Lecturer Use Only

Lecturer Comments:

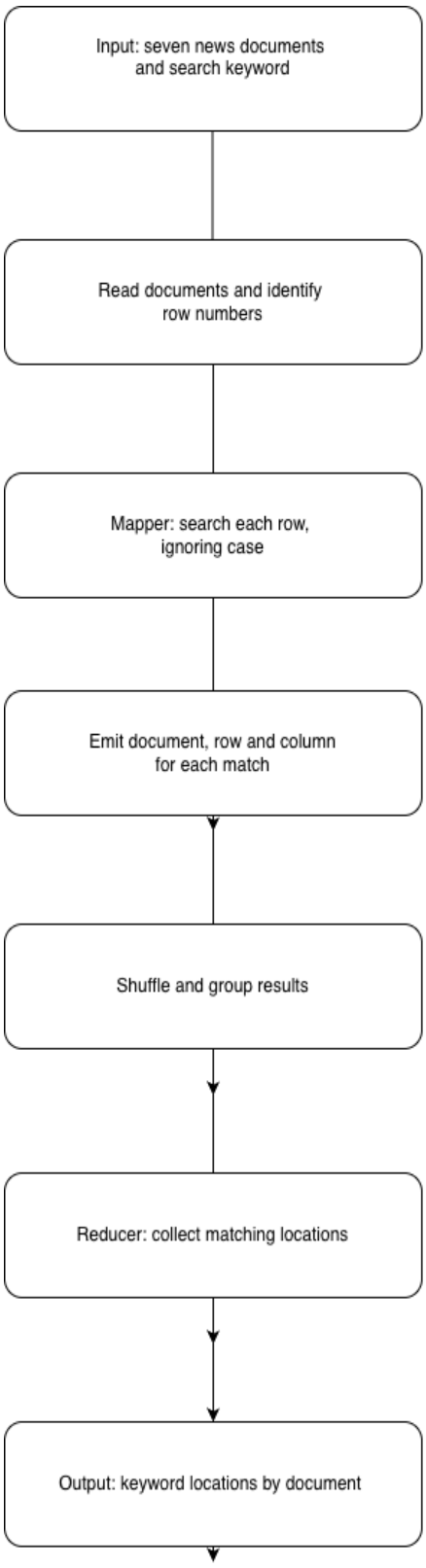
Marks Awarded (%):

Lecturer Signature:

Date:

Question 1: MapReduce Process

(1.1)



The proposed MapReduce diagram process searches a collection of news documents for a keyword supplied by the user. The search is case-insensitive and returns the document name, row number and starting column position of each occurrence.

1. Input and Configuration: The seven news documents are supplied as input to the Hadoop job. The user enters a search keyword, such as “africa”, when running the program. The keyword is passed to the Mapper through the job configuration, allowing it to be changed without modifying the source code.

2. Reading the documents: The input process identifies each document and its rows. Row numbers begin at 1, and blank lines are included in the count. This allows each occurrence to be located within its original document.

3. Map phase: The Mapper examines the document text for the supplied keyword without distinguishing between the uppercase and lowercase letters. For every occurrence found, it records the document name, row number and starting column position. Columns are counted from 1 and represent character positions within the row.

4.Intermediate output: The Mapper emits each matching locations as an intermediate key-value pair. The document name is used as the key, while the row and column form the associated value. For example, a match in doc1.txt could for instance be represented as:

(doc1.txt, Row 1, Column 64)

5. Shuffle and grouping: Hadoop groups the intermediate values that share the same document name. This brings together all matching locations for a particular document before they are passed to the Reducer.

6. Reduce phase: The Reducer receives each document name and its associated matching locations. It collects these locations and writes them to the final output.

7. Final output: The results show the document and the row and column of each keyword occurrence. Documents with no matches do not produce matching locations. For example, s search for ‘africa” would identify “Africa” in doc1.txt at row 1, column 64.

The implementation must account for the fact that Hadoop’s standard line input provides a byte offset rather than an actual row number. Therefore, the program will need to determine the correct row numbers explicitly.

Questions 1.2 – 1.4 (Copied code from the terminal)

```
import java.io.IOException;

import java.util.regex.Matcher;

import java.util.regex.Pattern;


import org.apache.hadoop.conf.Configuration;

import org.apache.hadoop.fs.Path;

import org.apache.hadoop.io.LongWritable;

import org.apache.hadoop.io.Text;

import org.apache.hadoop.mapreduce.Job;

import org.apache.hadoop.mapreduce.Mapper;

import org.apache.hadoop.mapreduce.Reducer;

import org.apache.hadoop.mapreduce.lib.input.FileInputFormat;

import org.apache.hadoop.mapreduce.lib.input.FileSplit;

import org.apache.hadoop.mapreduce.lib.input.TextInputFormat;

import org.apache.hadoop.mapreduce.lib.output.FileOutputFormat;


public class KeywordSearch {


    /*

     * Keep each document in a single input split.

     * This allows the Mapper to count rows from the beginning of each
file.

     */

    public     static     class     WholeFileTextInputFormat     extends
TextInputFormat {

        @Override

        protected boolean isSplitable(

                org.apache.hadoop.mapreduce.JobContext context,
```

```
            Path filename) {
        return false;
    }
}
```

// Question 1.2: Mapper

```
public static class SearchMapper
        extends Mapper<LongWritable, Text, Text, Text> {

    private Pattern keywordPattern;
    private String documentName;
    private long rowNumber;

    @Override
    protected void setup(Context context) throws IOException {
        String keyword = context.getConfiguration().get("search.keyword");

        if (keyword == null || keyword.trim().isEmpty()) {
            throw new IOException("A search keyword must be provided.");
        }

        // Treat the keyword as literal text and ignore letter case.
        keywordPattern = Pattern.compile(
                Pattern.quote(keyword),
                Pattern.CASE_INSENSITIVE | Pattern.UNICODE_CASE
        );
```

```
            FileSplit split = (FileSplit) context.getInputSplit();
            documentName = split.getPath().getName();
            rowNumber = 0;
        }

        @Override
        protected void map(LongWritable offset, Text value, Context
context)
                throws IOException, InterruptedException {

            rowNumber++;
            String line = value.toString();
            Matcher matcher = keywordPattern.matcher(line);

            while (matcher.find()) {
                // Count Unicode characters before the match, starting
at 1.
                int column = line.codePointCount(0, matcher.start()) +
1;

                context.write(
                        new Text(documentName),
                        new Text("Row " + rowNumber + ", Column " +
column)
                );
            }
        }
    }
```

// Question 1.3: Reducer

```
public static class SearchReducer
        extends Reducer<Text, Text, Text, Text> {

    @Override
    protected void reduce(Text document, Iterable<Text> locations,
                          Context context)
            throws IOException, InterruptedException {

        for (Text location : locations) {
            context.write(document, location);
        }
    }
}
```

// Question 1.4: Hadoop job configuration

```
public static void main(String[] args) throws Exception {

    if (args.length != 3 || args[2].trim().isEmpty()) {
        System.err.println(
                "Usage: KeywordSearch <input> <output> <keyword>"
        );
        System.exit(1);
    }

    Configuration config = new Configuration();
    config.set("search.keyword", args[2]);
```

```
        Job job = Job.getInstance(config, "Case-insensitive keyword
search");

        job.setJarByClass(KeywordSearch.class);


        job.setInputFormatClass(WholeFileTextInputFormat.class);

        job.setMapperClass(SearchMapper.class);

        job.setReducerClass(SearchReducer.class);


        job.setMapOutputKeyClass(Text.class);

        job.setMapOutputValueClass(Text.class);

        job.setOutputKeyClass(Text.class);

        job.setOutputValueClass(Text.class);


        FileInputFormat.addInputPath(job, new Path(args[0]));

        FileOutputFormat.setOutputPath(job, new Path(args[1]));


        System.exit(job.waitForCompletion(true) ? 0 : 1);

    }

}
```

```
(base) cristinaabrahamse@Cristinas-MacBook-Pro-2 Question1 % cat output_africa/part-r-00000
```

```
doc1.txt        Row 3, Column 95
doc1.txt        Row 3, Column 11
doc1.txt        Row 1, Column 64
doc2.txt        Row 58, Column 81
doc2.txt        Row 58, Column 5
doc2.txt        Row 33, Column 24
doc2.txt        Row 3, Column 32
doc2.txt        Row 1, Column 57
doc3.txt        Row 54, Column 44
doc3.txt        Row 52, Column 86
doc3.txt        Row 50, Column 142
doc3.txt        Row 48, Column 112
doc3.txt        Row 46, Column 7
doc3.txt        Row 40, Column 146
doc3.txt        Row 5, Column 7
doc3.txt        Row 3, Column 7
doc3.txt        Row 1, Column 38
doc5.txt        Row 48, Column 217
doc5.txt        Row 38, Column 27
doc5.txt        Row 23, Column 121
doc5.txt        Row 3, Column 88
doc5.txt        Row 1, Column 62
```

The Hadoop MapReduce job was executed using “africa” as the search keyword. The job completed successfully and returned 22 matching locations across four of the seven input documents. Each result identifies the document name, row number and starting column position of the match. The search was case-insensitive, and the keyword was supplied when the program was run.

```
doc1.txt        Row 55, Column 52
doc1.txt        Row 55, Column 7
doc1.txt        Row 52, Column 59
doc1.txt        Row 50, Column 1
doc1.txt        Row 49, Column 67
doc1.txt        Row 46, Column 110
doc1.txt        Row 44, Column 187
doc1.txt        Row 35, Column 132
doc1.txt        Row 33, Column 135
doc1.txt        Row 25, Column 110
doc1.txt        Row 25, Column 66
doc1.txt        Row 19, Column 61
doc1.txt        Row 17, Column 134
doc1.txt        Row 15, Column 79
doc1.txt        Row 15, Column 49
doc1.txt        Row 11, Column 144
doc1.txt        Row 11, Column 52
doc1.txt        Row 9, Column 1
doc1.txt        Row 7, Column 140
doc1.txt        Row 5, Column 12
doc1.txt        Row 3, Column 161
doc1.txt        Row 3, Column 103
doc1.txt        Row 1, Column 45
```

Figure 1: Results of the MapReduce keyword search using the dynamically supplied keyword “tax”.

The program was tested using two different search keywords: “africa” and another example “tax”. The first search returned 22 matching locations across four documents, while the second returned 23 matching locations in doc1.txt. These texts demonstrate that they keyword can be supplied at runtime without modifying the program. The search is case insensitive and identifies the keyword when it occurs within a longer word.

```
doc1.txt        Row 1, Column 45
(base) cristinaabrahamse@Cristinas-MacBook-Pro-2 Question1 % hadoop jar KeywordSearch.jar KeywordSearch map_reduce_test_docs outpu
t_upper AFRICA
```

```
                Bytes Written=801
(base) cristinaabrahamse@Cristinas-MacBook-Pro-2 Question1 % diff <(sort output_africa/part-r-00000) <(sort output_upper/part-r-00
000)
(base) cristinaabrahamse@Cristinas-MacBook-Pro-2 Question1 %
```

Case-insensitivity test: The program was run with the uppercase keyword “AFRICA”, and its results were compared with those from the lowercase keyword “africa”. The comparison returned no differences, confirming that both searches produced the same matching locations.

Question 2: Latent Dirichlet Allocation (LDA)

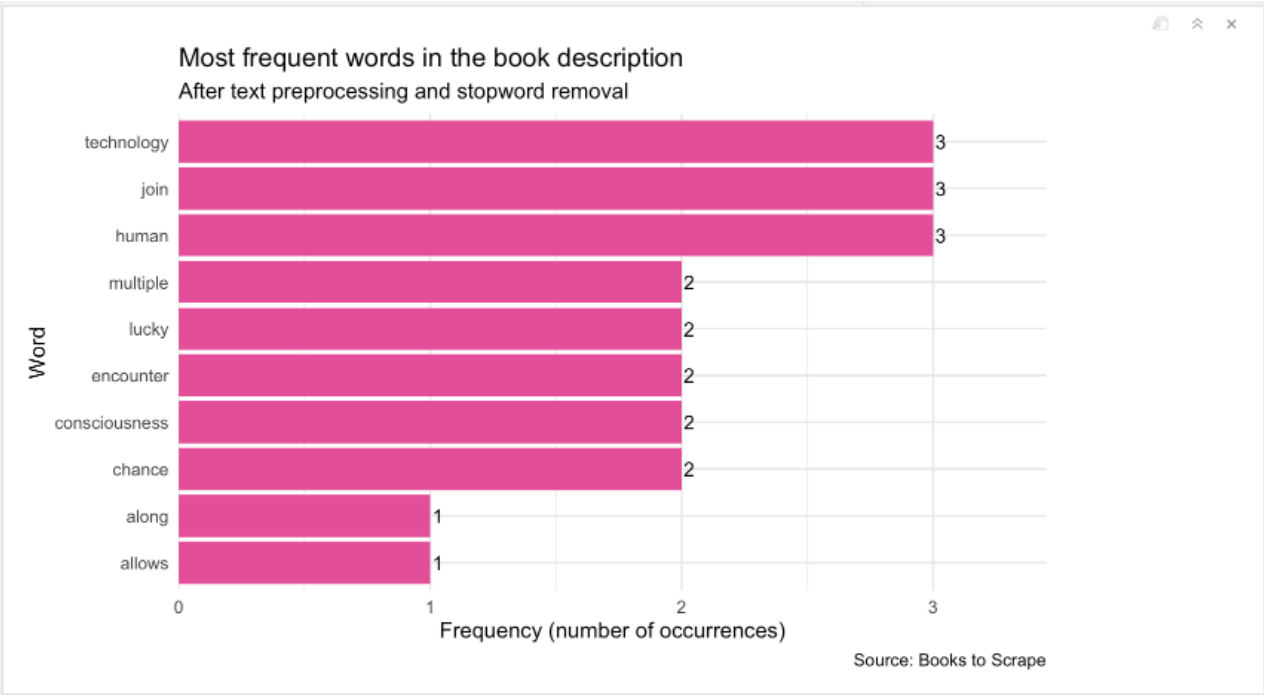
Question 2 was completed in RStudio using an R Notebook. The notebook documents the loading and preprocessing of the news articles, the Latent Dirichlet Allocation (LDA) topic modelling, and the visualisations and interpretation of the results. The R Notebook and associated dataset are available in the project’s GitHub repository.

Question 3: Webscraping & Text Summarisation

The R Notebook contains the R scripts, outputs and explanations for Questions 3.1and 3.2, including the extraction of the book description from the selected website, the creation of an R data structure and the preprocessing of the collected text.

For Question 3.3, the R notebook contains the scripts and outputs for the text summarisation techniques. The visualisations and their findings are also presented below.

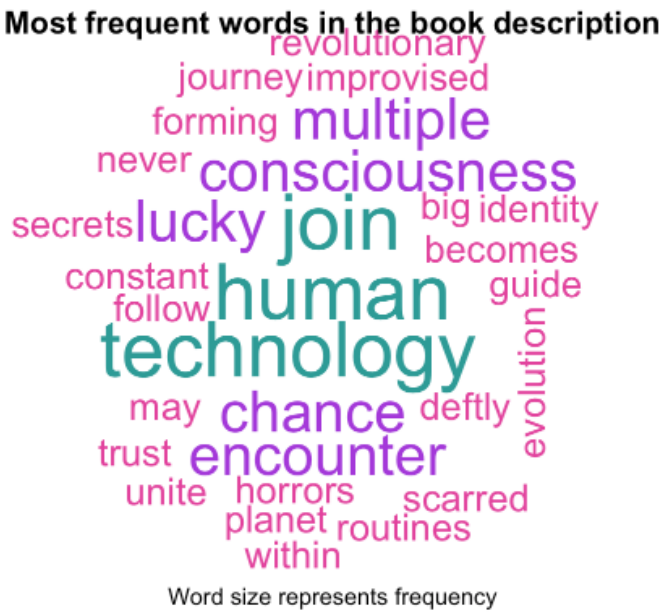
Question 3.3: Text Summarisation and Visualisation



Findings:

The pink bar chart above presents the ten most frequent words in the book description after text preprocessing and stopword removal. The words “technology”, “join” and “human” each appear three times, making them the most frequent words in the cleaned text. The words “consciousness”, “chance” and “lucky” each appear twice.

The results suggest that the description focuses on technology and human consciousness, as well as the experience of the two main characters. The book description is relatively short, so the frequencies are low and should be interpreted in the context of this single text.



Findings:

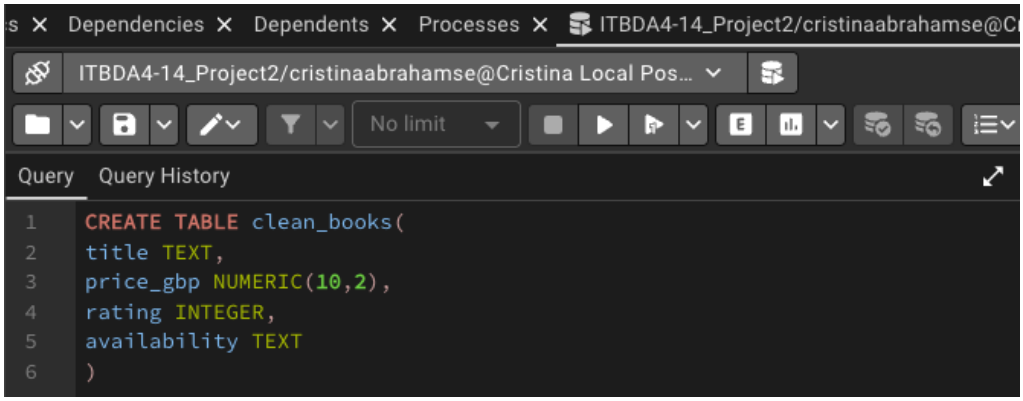
The above word cloud presents the most frequent words in the cleaned description of Join. Larger words appear more frequently in the text. The words “human”, “technology”, and “join” are among the largest, while “consciousness”, “chance” and “lucky” are also visible.

These words highlight the descriptions focus on technology, human consciousness and experiences of the two main characters. The word cloud complements the bar chart by providing a visual overview of the vocabulary, while the bar chart shows the exact frequencies. Both visualisations are based on the same short book description.

Question 4: Web scraping & PostgreSQL

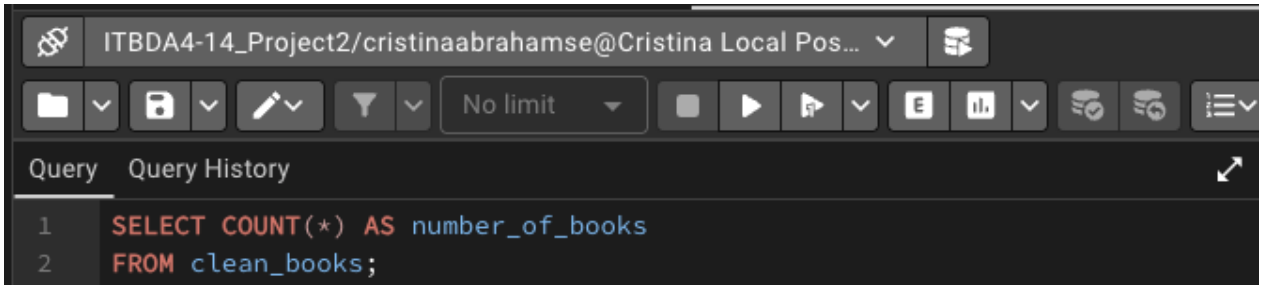
Questions 4.1 and 4.2 were completed in RStudio using an R Notebook. The notebook documents the extraction of book records from the selected website, the use of regular expressions, the cleaning and assessment of the data, and the export of the cleaned dataset as a CSV file. The R Notebook and dataset are available in the project's GitHub repository.

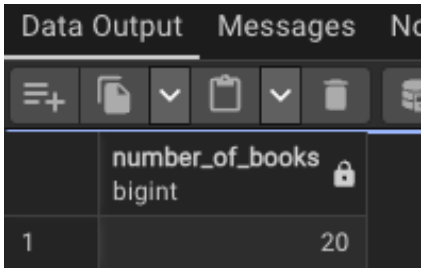
(4.3) Creation of table called clean_books inside PostgreSQL





After importing the cleaned CSV file into PostgreSQL, the following query was used to verify the number of records in the clean_books table. The result confirms that 20 book records were imported successfully.





Investigation 1: Average Price by Rating

```sql
-- Compare the number of books and average price for each rating
SELECT
    rating,
    COUNT(*) AS number_of_books,
    ROUND(AVG(price_gbp), 2) AS average_price_gbp
FROM clean_books
GROUP BY rating
ORDER BY rating;
```

Data Output | Messages | Notifications

	rating integer	number_of_books bigint	average_price_gbp numeric
1	1	6	40.02
2	2	3	36.83
3	3	3	42.32
4	4	4	31.11
5	5	4	39.75

The first SQL query grouped the 20 books by their star rating and calculated the number of books and average price for each group. Books rated three stars had the highest average price (£42.32), based on three books. Books rated four stars had the lowest average price of (£31.11), based on four books. Te results do not show a consistent increase in average price as the rating increases.

These findings apply only to the 20 selected listings. The website's prices and ratings are simulated, so they should not be interpreted as real market prices or reader reviews.

Investigation 2: Most expensive books

ITBDA4-14_Project2/cristinaabrahamse@Cristina Local Pos... | No limit

Query | Query History

```sql
-- I'm identifying the five most expensive books in my dataset
SELECT
    title,
    price_gbp,
    rating
FROM clean_books
ORDER BY price_gbp DESC
LIMIT 5;
```

Data Output | Messages | Notifications

Showing rows: 1 to 5 | Page No: 1

	title text	price_gbp numeric (10,2)	rating integer
1	Our Band Could Be Your Life: Scenes from the American Indie Underground, 1981-1...	57.25	3
2	Sapiens: A Brief History of Humankind	54.23	5
3	Tipping the Velvet	53.74	1
4	Scott Pilgrim's Precious Little Life (Scott Pilgrim #1)	52.29	5
5	The Black Maria	52.15	1

The second SQL query sorted the books by price in descending order and displayed the five most expensive titles. Our Band Could Be Your Life was the most expensive at £57.25, followed by Sapiens: A Brief History of Humankind at £54.23 and Tipping the Velvet at £53.74. The five books had different ratings, ranging from one to five stars, so the highest priced books did not all have the same rating.

These results describe the 20 scraped listings. The website is a demonstration site; a its prices and ratings are simulated.

(4.4) The project code, notebooks, datasets and supporting files are available at Github Link: https://github.com/CristinaAbEdu/ITBDA4-14-Assignment2_Cristina_Abrahamse

Reference list

1. Apache Hadoop Project (n.d.). *MapReduce tutorial*. [online] Hadoop 1.2.1 Documentation. Available at: https://hadoop.apache.org/docs/current1/mapred_tutorial.html [Accessed 5 Nov. 2026].
2. books.toscrape.com. (n.d.). *All products | books to scrape - sandbox*. [online] Available at: https://books.toscrape.com/ [Accessed 5 Nov. 2026].
3. Github.io. (2024). *Text mining using DPLYR, GGPLOT2, and other tidy tools*. [online] Available at: https://juliasilge.github.io/tidytext/ [Accessed 5 Nov. 2026].
4. Grün, B. and Hornik, K. (2011). Topicmodels: An R package for fitting topic models. *Journal of Statistical Software*, 40(13). doi:10.18637/jss.v040.i13.
5. Mihalcea, R. and Tarau, P. (2004). *TextRank: Bringing order into texts*. [online] Available at: https://aclanthology.org/W04-3252.pdf [Accessed 5 Nov. 2026].
6. PostgreSQL Documentation. (2026). *2.7. Aggregate functions*. [online] Available at: https://www.postgresql.org/docs/current/tutorial-agg.html [Accessed 5 Nov. 2026].
7. rvest.tidyverse.org. (n.d.). *Web scraping 101*. [online] Available at: https://rvest.tidyverse.org/articles/rvest.html [Accessed 5 Nov. 2026].
8. Silge, J. (2019). *Text mining with R*. [online] Tidytextmining.com. Available at: https://www.tidytextmining.com/ [Accessed 5 Nov. 2026].
9. stringr.tidyverse.org. (n.d.). *Regular expressions*. [online] Available at: https://stringr.tidyverse.org/articles/regular-expressions.html [Accessed 5 Nov. 2026].
10. https://books.toscrape.com/catalogue/join_902/index.html

AI Disclosure Appendix

AI tool used	Purpose / how it was used	How I verified or adapted the output
e.g. ChatGPT 4o	e.g. Brainstormed initial argument outline	e.g. Cross-checked with three peer-reviewed sources; fully rewritten in own words
e.g. Claude Opus 4.7	e.g. Brainstormed initial argument outline	e.g. Cross-checked with three peer-reviewed sources; fully rewritten in own words
e.g. Grok 4	e.g. Brainstormed initial argument outline	e.g. Cross-checked with three peer-reviewed sources; fully rewritten in own words

Declaration:

I/we declare that this assessment is my/our own original work, except for source material explicitly acknowledged. I/we confirm that any use of Artificial Intelligence (AI) tools has been applied only as permitted by the assessment instructions and has been fully disclosed where required.

I/we understand that I/we remain responsible for the accuracy, originality, and academic quality of this submission. I/we am/are aware of the institution's policies on academic integrity and the requirements relating to the responsible use of AI in assessments.

I/we acknowledge that failure to comply with these requirements, including undeclared or inappropriate AI use, may result in academic penalties or disciplinary procedures in accordance

with institutional policy.

Student Signature(s)	Date